Lesson 13 Sidebars

A Hit That Reacts

VIDEO GAME CREATION

These aren't part of the lesson's required work. Think of them the way a textbook sidebar works: a deeper, tangential look at something this lesson's build actually ran into - why one approach was chosen over the others, what the alternatives were, or an idea worth a longer look than the Handout has room for. Pull one out for a discussion moment if it fits, hand it to a curious student, or skip it entirely - nothing here is required reading.

The White Flash That Wasn't (Part Two of the Color Story)

Lesson 2's color sidebar explained why tinting a colored sprite comes out muddy. This lesson's hit-flash effect ran into the exact same root cause - except this time, the result wasn't muddy. It was invisible.

A color tint is a multiply operation: result = original color × tint color, channel by channel. Multiplying by WHITE specifically leaves any color completely unchanged, since multiplying by 1 (white, in normalized color terms) is a no-op. So tinting the enemy white for a hit-flash doesn't just come out muddy on a colored enemy sprite the way Lesson 2's Recolor did - it produces zero visible change at all, since the enemy sprite wasn't white to begin with.

That wasn't hypothetical here: three of eight engines' Level 1 solutions were originally written to set the enemy's tint to white on hit (Godot's modulate, Unity's spriteRenderer.color, GameMaker's image_blend) - all three would have silently done nothing. Easy to miss during a quick test, too, since the enemy still gets destroyed a moment later either way; the missing flash doesn't look obviously broken, it just isn't there.

The fix is the same fix as Lesson 2's Recolor: swap in a genuinely white pre-made sprite instead of tinting toward white. TIC-80's flash was never affected, because it was already built as a sprite-swap from the very start - drawing a dedicated white "flash" version of the enemy directly in the Sprite Editor - the same pattern that made TIC-80 the one engine that never needed Lesson 2's Recolor fix either.

The broader point worth sitting with: this is exactly why "why does this work the way it works" matters more than "does this look right today." The same underlying limitation - a multiply-tint's total dependence on the source art's own color - surfaced twice, in two different lessons, wearing two different-looking symptoms (muddy color, then no visible change at all), because it was the same rule both times. Actually understanding the rule the first time means being able to predict where else it's going to bite - which is a big part of what separates debugging by understanding from debugging by guessing.